Name – Armaan Patnaik
Class – ly 21
roll no. - 2223448

HPC LAB Assignment 4

Q.1 Process 0 initializes an N times N matrix A and a vector x of size N. Assume N is divisible by the number of MPI processes. Distribute consecutive rows of matrix A among the processes using MPI_Scatter. Unit vector x is available to every process. Each process computes its portion of

y=Ax.

Use MPI_Allgather so that, after the computation, every process contains the complete result vector y. Each process should then calculate the maximum value in y, and verify using MPI_Allreduce that all processes obtain the same global maximum. Process 0 should print A, x, and the final result vector.

Code-

```
/*
 * matvec.c  -  y = A x  using MPI_Scatter, MPI_Bcast, MPI_Allgather, MPI_Allreduce
 *
 * Compile : mpicc -o matvec matvec.c
 * Run     : mpirun --oversubscribe -np 4 ./matvec [N]      (default N = 8)
 *           N must be divisible by the number of processes.
 */
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, size, N = 8;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc > 1) N = atoi(argv[1]);

    if (N <= 0 || N % size != 0) {
```

Name – Armaan Patnaik
Class – ly 21
roll no. - 2223448

```
    if (rank == 0)
        fprintf(stderr, "Error: N (%d) must be positive and divisible by "
                "the number of processes (%d)\n", N, size);
    MPI_Finalize();
    return 1;
  }

  int rows = N / size;                  /* rows of A per process */

  double *A      = NULL;                /* full matrix: rank 0 only */
  double *x      = malloc(N * sizeof(double));
  double *y      = malloc(N * sizeof(double));       /* full result     */
  double *A_local = malloc(rows * N * sizeof(double)); /* my block of rows */
  double *y_local = malloc(rows * sizeof(double));    /* my part of y    */

  /* ---------- Process 0 initialises A and x ---------- */
  if (rank == 0) {
    A = malloc(N * N * sizeof(double));
    for (int i = 0; i < N; i++)
      for (int j = 0; j < N; j++)
        A[i * N + j] = i * N + j + 1;    /* 1, 2, 3, ... */

    /* unit vector e_0 = (1, 0, 0, ..., 0)
       (for a normalised vector use: x[i] = 1.0 / sqrt(N) and link with -lm) */
    for (int i = 0; i < N; i++) x[i] = 0.0;
    x[0] = 1.0;
  }

  /* x must be available on every process */
  MPI_Bcast(x, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

Name – Armaan Patnaik
Class – ly 21
roll no. - 2223448

```
  /* consecutive blocks of 'rows' rows go to each process */
  MPI_Scatter(A, rows * N, MPI_DOUBLE,
              A_local, rows * N, MPI_DOUBLE,
              0, MPI_COMM_WORLD);

  /* ---------- local part of y = A x ---------- */
  for (int i = 0; i < rows; i++) {
     y_local[i] = 0.0;
     for (int j = 0; j < N; j++)
        y_local[i] += A_local[i * N + j] * x[j];
  }

  /* everyone receives the complete y */
  MPI_Allgather(y_local, rows, MPI_DOUBLE,
                y,       rows, MPI_DOUBLE,
                MPI_COMM_WORLD);

  /* ---------- maximum of y on each process ---------- */
  double local_max = y[0];
  for (int i = 1; i < N; i++)
     if (y[i] > local_max) local_max = y[i];

  /* verify that all processes obtained the same maximum */
  double global_max;
  MPI_Allreduce(&local_max, &global_max, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);

  int agree = (local_max == global_max);      /* does MY value match? */
  int all_agree;
  MPI_Allreduce(&agree, &all_agree, 1, MPI_INT, MPI_LAND, MPI_COMM_WORLD);

  /* ---------- Process 0 prints everything ---------- */
```

Name – Armaan Patnaik
Class – ly 21
roll no. - 2223448

```
    if (rank == 0) {

        printf("Number of processes = %d, N = %d\n\n", size, N);


        printf("Matrix A:\n");

        for (int i = 0; i < N; i++) {

            for (int j = 0; j < N; j++)

                printf("%7.1f", A[i * N + j]);

            printf("\n");

        }


        printf("\nVector x:\n");

        for (int i = 0; i < N; i++) printf("%7.1f", x[i]);


        printf("\n\nResult vector y = A x:\n");

        for (int i = 0; i < N; i++) printf("%7.1f", y[i]);


        printf("\n\nMaximum value in y (local) = %.1f\n", local_max);

        printf("Maximum value in y (global, MPI_Allreduce) = %.1f\n", global_max);

        printf("All processes agree on the maximum: %s\n",

              all_agree ? "YES" : "NO");

    }


    free(x); free(y); free(A_local); free(y_local);

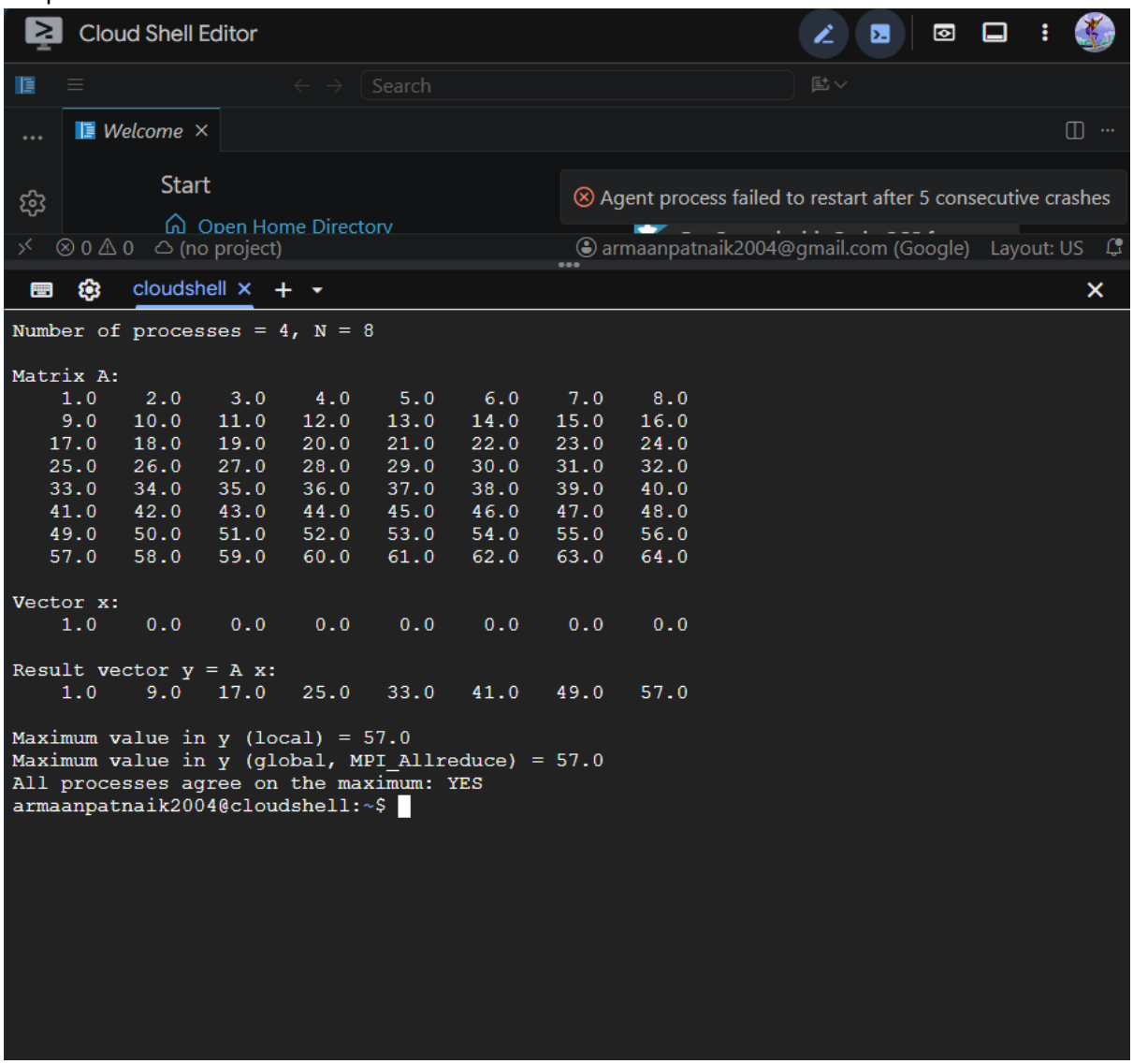
    if (rank == 0) free(A);


    MPI_Finalize();

    return 0;

}
```

Name – Armaan Patnaik
Class – ly 21
roll no. - 2223448
Output -



Name – Armaan Patnaik
Class – ly 21
roll no. - 2223448
Question 2. - Define the following structure:

```
struct Student {
   int roll_no;
   char name[20];
   double marks;
};
```

Process 0 initializes records for (N) students. Create an MPI derived datatype corresponding to Student using MPI_Get_address, MPI_Type_create_struct, and MPI_Type_commit. Distribute equal numbers of student records among the processes. Each process should determine the student with the highest marks in its local block. Determine the overall highest mark using an MPI collective operation, and identify which process contains that student. Finally, send the complete record of the highest-scoring student to Process 0 and display the student's roll number, name, and marks. Free the derived datatype before terminating the program.

Code -

```
/*
 * student.c  -  MPI derived datatype for struct Student
 *
 * Compile : mpicc -o student student.c
 * Run     : mpirun --oversubscribe -np 4 ./student [N]     (default N = 8)
 *           N (number of students) must be divisible by the number of processes.
 */
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

struct Student {
   int    roll_no;
   char   name[20];
   double marks;
};

int main(int argc, char *argv[])
```

Name – Armaan Patnaik
Class – ly 21
roll no. - 2223448

```
{
   int rank, size, N = 8;

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   if (argc > 1) N = atoi(argv[1]);

   if (N <= 0 || N % size != 0) {
      if (rank == 0)
         fprintf(stderr, "Error: N (%d) must be positive and divisible by "
                         "the number of processes (%d)\n", N, size);
      MPI_Finalize();
      return 1;
   }

   int per_proc = N / size;

   /* ---------- build the derived datatype ---------- */
   struct Student tmp;
   MPI_Aint base, a_roll, a_name, a_marks;

   MPI_Get_address(&tmp,         &base);
   MPI_Get_address(&tmp.roll_no,  &a_roll);
   MPI_Get_address(&tmp.name,     &a_name);
   MPI_Get_address(&tmp.marks,    &a_marks);

   int         blocklens[3] = { 1, 20, 1 };
   MPI_Aint    displs[3]    = { a_roll - base, a_name - base, a_marks - base };
   MPI_Datatype types[3]    = { MPI_INT, MPI_CHAR, MPI_DOUBLE };
```

Name – Armaan Patnaik
Class – ly 21
roll no. - 2223448

```
  MPI_Datatype MPI_STUDENT;


  MPI_Type_create_struct(3, blocklens, displs, types, &MPI_STUDENT);
  MPI_Type_commit(&MPI_STUDENT);


  /* ---------- Process 0 creates the records ---------- */
  struct Student *all   = NULL;
  struct Student *local = malloc(per_proc * sizeof(struct Student));


  if (rank == 0) {
    all = malloc(N * sizeof(struct Student));
    srand(42);
    for (int i = 0; i < N; i++) {
      all[i].roll_no = 101 + i;
      snprintf(all[i].name, sizeof(all[i].name), "Student_%d", i + 1);
      all[i].marks = 35.0 + (rand() % 651) / 10.0;     /* 35.0 .. 100.0 */
    }


    printf("Number of processes = %d, number of students = %d\n\n", size, N);
    printf("%-8s %-12s %-6s\n", "Roll", "Name", "Marks");
    for (int i = 0; i < N; i++)
      printf("%-8d %-12s %-6.1f\n", all[i].roll_no, all[i].name, all[i].marks);
    printf("\n");
  }


  /* equal share of records for every process */
  MPI_Scatter(all,   per_proc, MPI_STUDENT,
          local, per_proc, MPI_STUDENT,
          0, MPI_COMM_WORLD);


  /* ---------- best student in my local block ---------- */
```

Name – Armaan Patnaik
Class – ly 21
roll no. - 2223448

```
    int best = 0;
    for (int i = 1; i < per_proc; i++)
        if (local[i].marks > local[best].marks) best = i;

    /* ---------- overall highest mark + owning process ----------
       MPI_MAXLOC on (value, rank) pairs gives both in one collective.
       On a tie it returns the smallest rank.                       */
    struct { double marks; int rank; } in, out;
    in.marks = local[best].marks;
    in.rank  = rank;
    MPI_Allreduce(&in, &out, 1, MPI_DOUBLE_INT, MPI_MAXLOC, MPI_COMM_WORLD);

    /* ---------- send full record of the topper to process 0 ---------- */
    struct Student topper;

    if (out.rank == 0) {
        if (rank == 0) topper = local[best];        /* already on process 0 */
    } else if (rank == out.rank) {
        MPI_Send(&local[best], 1, MPI_STUDENT, 0, 0, MPI_COMM_WORLD);
    } else if (rank == 0) {
        MPI_Recv(&topper, 1, MPI_STUDENT, out.rank, 0, MPI_COMM_WORLD,
                 MPI_STATUS_IGNORE);
    }

    if (rank == 0) {
        printf("Highest marks = %.1f (found on process %d)\n", out.marks, out.rank);
        printf("Roll number : %d\n", topper.roll_no);
        printf("Name        : %s\n", topper.name);
        printf("Marks       : %.1f\n", topper.marks);
    }
```

Name – Armaan Patnaik
Class – ly 21
roll no. - 2223448

```
    /* ---------- clean up ---------- */

    MPI_Type_free(&MPI_STUDENT);

    free(local);

    if (rank == 0) free(all);


    MPI_Finalize();

    return 0;

}
```

Output-

